
AutoBuild: A Boundary-Aligned Replay Protocol for Repository-Task Validity

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Automatically synthesized repository tasks are useful for training long-horizon
2 coding agents only when their scores measure the submitted repository rather
3 than hidden state accumulated during task construction or rollout. Fresh-container
4 grading removes this hidden-state dependence, but the required source stripping,
5 candidate sanitization, test restoration, and result parsing can themselves change
6 the task. We call these two failure modes *evaluation-path drift*. We introduce
7 **AutoBuild**, a boundary-aligned repository-to-task synthesis and validation frame-
8 work that verifies the complete path from a known-good repository to the deployed
9 grader. AutoBuild builds a reference environment, synthesizes the task and test
10 interface from shared executable evidence, constructs a source-free grading envi-
11 ronment, and performs two-stage boundary-aligned replay before admitting a task.
12 Its acceptance contract compares exact test identities and outcomes, fails closed
13 on execution or parsing errors, and rejects empty, stub, mutant, and adversarial
14 candidates. In controlled evaluation, full replay reduces invalid-control acceptance
15 from 14.6% to 0.0%. Under a matched 4.4B-token training budget, AutoBuild-
16 selected data improves NL2Repo-Bench Pass@1 from 18.9 to 41.5 for a 7B model
17 and from 22.3 to 47.3 for a 35B-A3B model; BeyondSWE-Doc2Repo mean test
18 pass rate improves from 29.2 to 53.2 and from 43.8 to 61.2, respectively. These
19 results show that validating the final evaluation path improves both task validity
20 and downstream repository-generation capability.

21 1 Introduction

22 Long-horizon coding agents increasingly operate at repository scale: they install dependencies,
23 inspect tests, edit multiple modules, and repeatedly execute the project before returning a final artifact.
24 Benchmarks such as NL2Repo-Bench and BeyondSWE-Doc2Repo evaluate this capability by asking
25 an agent to construct an entire repository from a natural-language specification [3, 2]. Progress in
26 this setting depends on scalable, executable training tasks, yet task synthesis is useful only when the
27 resulting reward faithfully measures the repository submitted by the agent.

28 Existing systems have made substantial progress on complementary parts of this problem. Re-
29 poLaunch automates repository build and test management across languages and platforms [5];
30 DeNovoSWE scales sandboxed document-to-repository task and trajectory construction [10]. How-
31 ever, making a source repository executable does not by itself guarantee that the *derived task* remains
32 valid after it is packaged and deployed. Repository-to-task construction introduces a second path—
33 source removal, test preservation, candidate export, overlay, collection, and score parsing—whose
34 behavior may differ from the path validated during repository setup.

35 This gap creates three challenges. First, grading inside an agent-modified workspace can reward
36 undeclared packages, caches, external files, environment variables, or background processes instead

37 of the delivered repository. Second, moving to a clean grader requires destructive transformations
38 that can remove valid implementation, retain reference code, or allow candidate-authored tests to
39 affect the score. Third, aggregate pass counts can hide test replacement, parser errors, or additional
40 failures; task validity therefore requires agreement at the level of test identities and outcomes, not
41 merely a nonzero or full scalar score.

42 We introduce **AutoBuild**, a boundary-aligned, test-grounded framework for synthesizing and val-
43 idating repository-level coding tasks. AutoBuild follows four stages. It first builds a reference
44 environment and extracts reproducible test commands and structured outcomes. It then synthesizes a
45 task specification and test interface from a shared repository snapshot. Next, it constructs a source-free
46 grading environment that receives only an allowlisted candidate artifact. Finally, it replays the same
47 known-good reference on both sides of the transformation boundary and admits the task only when
48 exact test identities and outcomes agree. Empty, stub, mutant, and adversarial candidates serve as
49 negative controls against false acceptance.

50 The design is motivated by a simple principle: every transformation that separates the validated
51 source repository from the deployed judge must itself be validated through the deployed path. We
52 call violations of this principle *evaluation-path drift*. AutoBuild operationalizes it as an Isolate-
53 Transform-Replay protocol and a strict acceptance contract.

54 We evaluate AutoBuild through three questions: whether clean grading removes dependence on
55 mutable development state, whether the two replay stages prevent invalid tasks from being admitted,
56 and whether the resulting data improves downstream training. Full replay reduces invalid-control
57 acceptance from 14.6% to 0.0%. With equal task volume, model, optimizer, schedule, and 4.4B-token
58 budget, AutoBuild-selected data improves NL2Repo-Bench Pass@1 by 22.6 and 25.0 percentage
59 points for 7B and 35B-A3B models, and BeyondSWE-Doc2Repo mean test pass rate by 24.0 and
60 17.4 points.

61 Our contributions are threefold:

- 62 • We formulate *evaluation-path drift*, distinguishing hidden-state dependence during grading
63 from validity errors introduced while transforming a repository into a task.
- 64 • We present AutoBuild, a boundary-aligned repository-to-task synthesis and validation
65 framework with clean grading, two-stage replay, exact identity-based acceptance, negative
66 controls, and cross-grader parity.
- 67 • We provide controlled before-after and component-ablation results showing that AutoBuild
68 eliminates invalid-control acceptance in our study and that its selected data improves
69 repository-generation performance under matched training budgets.

70 2 Related Work

71 **Repository-level coding benchmarks.** SWE-bench and its extensions evaluate issue resolution
72 in existing repositories [4]. Recent benchmarks broaden the setting to long-horizon repository
73 construction. NL2Repo-Bench gives an agent a requirements document and an empty workspace,
74 then evaluates the packaged library with upstream tests in a separate image [3]. BeyondSWE includes
75 Doc2Repo, a document-to-repository setting that restores evaluator tests and scores an exported
76 repository in a fresh container [2]. These benchmarks define the downstream capability measured
77 in our experiments; AutoBuild instead studies how executable training tasks and their graders are
78 constructed and validated.

79 **Executable repository environments.** Repository build systems aim to convert arbitrary source
80 snapshots into reproducible executable environments. RepoLaunch decomposes this process into
81 preparation, build, and management stages, producing reusable rebuild commands, test commands,
82 and structured test-log parsers across languages and platforms [5]. Rule-based and agentic environ-
83 ment builders are also used by SWE-bench derivatives and repository-repair datasets. AutoBuild
84 adopts this environment-first view but addresses the next boundary: whether source removal, task
85 packaging, candidate delivery, and the final scorer preserve the behavior established by the reference
86 build.

87 **Automated software-engineering data.** SWE-smith constructs working Python environments
 88 before synthesizing faults that break previously passing tests [8]. ScaleSWE coordinates environment,
 89 test, and problem-statement agents and retains tasks with the required buggy/patched transitions [9].
 90 DeNovoSWE profiles repository behavior, synthesizes documentation with a divide-and-conquer
 91 and critic–repair process, filters trajectories by difficulty, and evaluates transfer to NL2Repo-Bench
 92 and BeyondSWE-Doc2Repo [10]. These methods show how executable evidence can scale task and
 93 trajectory construction. AutoBuild is complementary: it treats the final evaluation path as an object
 94 that must be replayed and validated before synthesized data are admitted.

95 **Benchmark validity and lifecycle checks.** SWE-Bench+ studies solution leakage and weak tests
 96 in repository-repair tasks [1]. Auto Benchmark Audit identifies hidden environment dependencies,
 97 specification gaps, and brittle graders, then measures ranking changes after filtering [7]. Change2Task
 98 validates healthy, task, and restored repository states when transferring historical changes across
 99 revisions [6]. AutoBuild focuses on a distinct but related lifecycle failure: the known-good repository
 100 may pass before task construction yet fail, or be accepted spuriously, after source stripping, candidate
 101 sanitization, test collection, and parsing. Its two-stage replay aligns validation with the exact deployed
 102 judge rather than with an earlier build checkpoint.

103 3 AutoBuild

104 AutoBuild is a boundary-aligned repository-to-task synthesis and validation framework that converts
 105 a fixed repository snapshot into a natural-language task, a development environment, and a deployed
 106 grader. Unlike pipelines that stop after confirming that the source repository builds, AutoBuild
 107 validates the complete transformation from the reference implementation to the candidate artifact
 108 evaluated by the judge. This section first formalizes the target property and then describes the four
 109 pipeline stages.

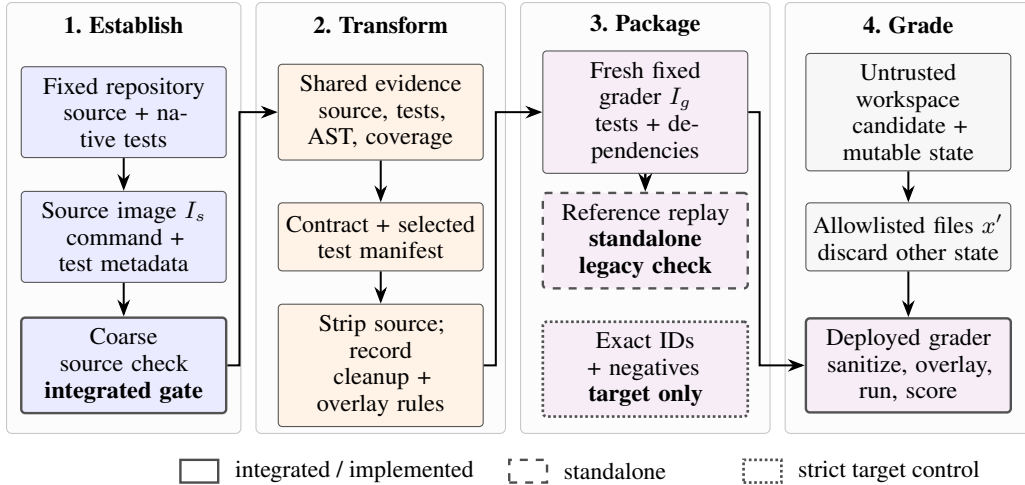


Figure 1: AutoBuild validates the complete repository-to-task path. A reference repository is built once, transformed into a task and a source-free grader, and replayed through the same packaging and judging path used for agent submissions.

110 3.1 Problem Formulation

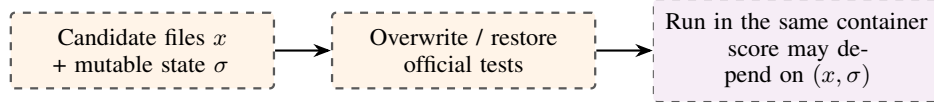
111 Let $r = (S, \mathcal{T}, c)$ denote a reference implementation, its native test suite, and a fixed commit. A
 112 coding trajectory η produces a mutable workspace

$$W_\eta = (x_\eta, \sigma_\eta), \quad (1)$$

113 where x_η is the candidate repository and σ_η contains all other accumulated state, including installed
 114 software, caches, configuration, external files, and processes. An in-place evaluator computes

$$J_{\text{reuse}}(W_\eta) = \phi(\text{Run}[W_\eta \oplus \mathcal{T}, C]), \quad (2)$$

(A) Stateful reuse



(B) Artifact-only clean room

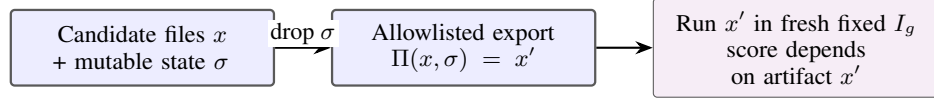


Figure 2: In-place test restoration retains undeclared development state. AutoBuild exports only the candidate repository to a fresh grader and separately validates that the repository-to-grader transformation preserves the reference behavior.

so two identical candidate repositories may receive different scores when their hidden states differ. A synthesized task is $z = (I_d, I_g, d, C, H, \mathcal{I}, \phi)$, comprising development and grading images, task description, test command, candidate export policy, intended test identities, and scorer. AutoBuild evaluates only the exported artifact in fixed grader state:

$$J_z(W_\eta) = \phi(\text{Run}[I_g \oplus \text{Export}_H(x_\eta), C]). \quad (3)$$

The first requirement is *artifact determinacy*: identical exported artifacts receive identical scores under a frozen task contract.

Isolation alone is insufficient because constructing I_g , applying H , restoring tests, and parsing outcomes may change behavior. Let $\mathbf{y}_s(S; \mathcal{I})$ and $\mathbf{y}_g(S; \mathcal{I})$ denote the per-test outcomes of the same known-good implementation before and after task transformation. In general,

$$\text{BuildPass}(S, I_d, C) \not\Rightarrow \text{GradePass}(S, I_g, H, C, \mathcal{I}, \phi). \quad (4)$$

AutoBuild therefore requires $\mathbf{y}_s = \mathbf{y}_g$ at the level of exact test identities and outcomes. We call dependence on σ_η , or disagreement introduced after the reference build, *evaluation-path drift*.

3.2 Reference Build and Test Interface

AutoBuild begins from a fixed Git repository URL and commit. Tool-using agents inspect dependency manifests and test configuration, install and build the project, execute the native test suite, and organize the successful interaction history into reproducible rebuild commands, test commands, and a parser that maps heterogeneous logs to per-test outcomes. The resulting development image I_d is the reference state for every downstream artifact.

A low-cost triage gate reloads I_d , executes the extracted commands, defines $T_s = P_s + F_s + E_s$, and requires

$$A_1(z; \tau) = \mathbf{1}[T_s > 0 \wedge P_s/T_s \geq \tau], \quad \tau = 0.9. \quad (5)$$

This aggregate check rejects clearly unusable snapshots early, but it is not the task-validity criterion. AutoBuild subsequently freezes the exact passing test-ID set and re-executes it with fail-closed process and parser checks.

3.3 Test-Grounded Task Synthesis

The task description and grader are derived from the same executable snapshot. AutoBuild extracts an abstract-syntax-tree inventory, profiles test coverage in I_d , and traces symbols referenced by the native tests. Public and tested-private symbols form a deterministic MUST-COVER set; when the public surface is large, a bounded tested-first subset preserves the interfaces that affect observable behavior.

A documentation agent uses repository browsing, test inspection, symbol retrieval, and runtime-signature tools to produce a from-scratch implementation contract. A bounded critic–repair loop

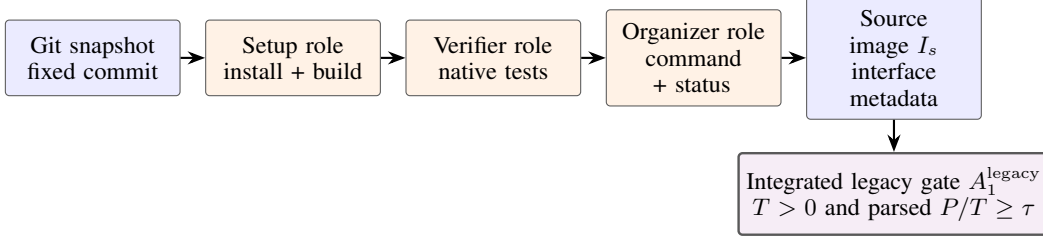


Figure 3: Reference-build stage. AutoBuild turns repository setup and test execution into reproducible commands and structured per-test outcomes, then freezes the known-good reference manifest.

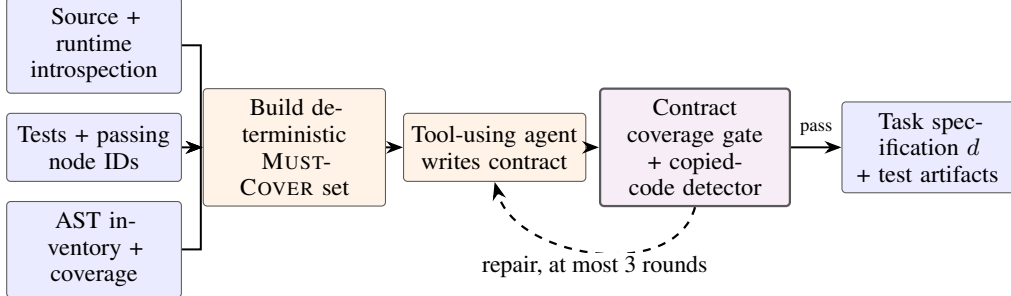


Figure 4: Test-grounded task synthesis. Source structure, runtime coverage, and test references provide shared evidence for the task contract; deterministic checks enforce interface coverage and prevent verbatim leakage.

145 checks that required symbols and interfaces are represented and rejects long normalized spans copied
 146 from source or tests. Tests used for construction are partitioned from a held-out behavioral split
 147 that is never exposed to the specification generator. This stage anchors the natural-language task to
 148 executable behavior without exposing the reference implementation at rollout time.

149 3.4 Source-Free Grading Environment

150 AutoBuild next separates development from grading. The clean-image builder preserves dependencies
 151 and evaluator-owned tests while removing the reference implementation, source-control metadata,
 152 installed-package copies, wheel caches, bytecode, and build remnants. It rejects an empty import
 153 surface and records immutable image, command, parser, and test-manifest digests.

154 At evaluation time, the candidate exporter admits only allowlisted repository paths. Before overlay,
 155 the grader removes explicit test paths and pytest-discoverable candidate tests, including `test_*.py`,
 156 `*_test.py`, and `conftest.py`; it then installs the candidate, runs the frozen command, and parses
 157 exact per-test outcomes. This clean boundary prevents a trajectory from receiving credit through
 158 packages, files, tests, or services left in its development workspace.

159 3.5 Two-Stage Boundary-Aligned Replay

160 The main quality gate compares a known-good implementation before and after task transformation.
 161 The source replay R_s executes the frozen test manifest in I_d . The grading replay R_g packages
 162 S exactly as an agent submission, sanitizes and overlays it into I_g , installs it, runs the deployed
 163 command, and invokes the deployed parser. A task is accepted only if

$$\mathcal{I}_s^c = \mathcal{I}_s^p = \mathcal{I}_g^c = \mathcal{I}_g^p = \mathcal{I}_m, \quad F_s = E_s = K_s = F_g = E_g = K_g = 0, \quad \text{rc}_s = \text{rc}_g = 0, \quad (6)$$

164 where c and p denote collected and passed test IDs, K denotes skipped tests under the declared policy,
 165 and $\mathcal{I}_m$ is the frozen manifest. Identity equality prevents a parser or collection change from replacing
 166 one intended test with another while preserving the same scalar count.

167 Positive replay establishes that one known-good implementation survives the transformation, but it
 168 cannot rule out a grader that accepts everything. AutoBuild therefore evaluates empty and import-only

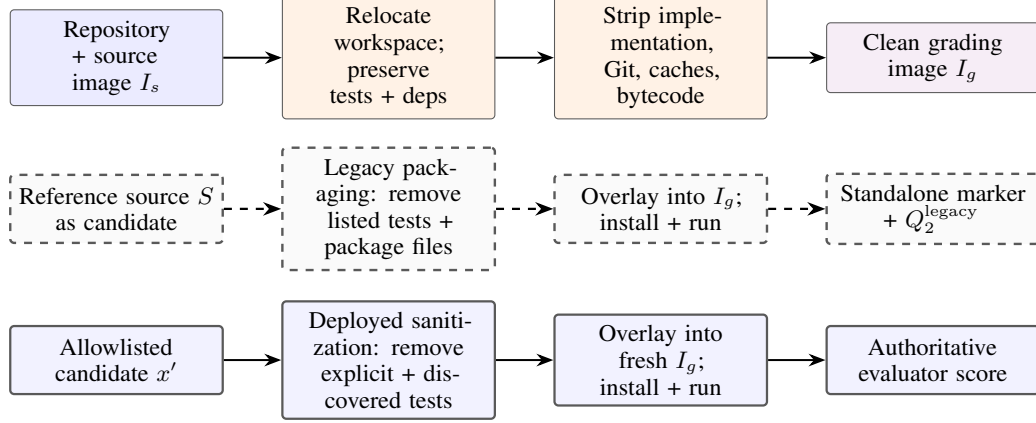


Figure 5: Source-free grading. Only the exported candidate enters a fixed image containing the declared runtime and evaluator-owned tests; the reference implementation is replayed through this same path before the task is admitted.

169 submissions, behavior stubs, source mutants, and adversarial workspaces containing candidate tests,
 170 hooks, packaging overrides, or source-retrieval attempts. Every negative control must score below
 171 the acceptance threshold. Finally, the same artifact and manifest must produce identical outcome
 172 vectors across delivered graders. Any installation, execution, parsing, identity, negative-control, or
 173 parity failure rejects the task.

174 4 Experiments

175 We organize the evaluation around three research questions. **RQ1** asks whether a fresh fixed grader
 176 removes dependence on mutable development state. **RQ2** asks whether the two replay stages and strict
 177 acceptance contract prevent invalid tasks from entering the dataset, and which stage contributes to the
 178 result. **RQ3** asks whether selecting data with AutoBuild improves downstream repository-generation
 179 models under a matched training budget.

180 4.1 Task-Validity Evaluation

181 For RQ1, each exported candidate is evaluated with identical official tests in both its agent-modified
 182 work container and a fresh fixed grader. We use benign candidates to measure behavioral agreement,
 183 controlled candidates that depend on undeclared packages, system libraries, runtime versions, en-
 184 vironment variables, caches, external files, or processes, and a natural candidate cohort to measure
 185 in-place-pass/fresh-fail transitions without injected faults.

186 For RQ2, we apply four acceptance policies to one frozen construction-complete cohort: no strict
 187 replay, source replay R_s only, grading replay R_g only, and both stages. Faults are injected into source
 188 stripping, candidate cleanup, overlay precedence, test collection, execution, and parsing. The primary
 189 metrics are *invalid-control escape*, the fraction of empty, stub, mutant, or adversarial candidates
 190 incorrectly accepted, and *valid-task retention*, the fraction of known-valid post-build tasks kept. We
 191 additionally report exact test-ID violations, negative-control rejection, and cross-grader agreement.

192 4.2 Downstream Training Evaluation

193 For RQ3, the baseline and AutoBuild arms are sampled from the same master pool and satisfy
 194 identical upstream feasibility requirements. They differ only in whether exact R_s/R_g , negative-
 195 control, and parity checks are used to select examples. We match task volume, supervised-token
 196 count within a predeclared 0.5% packing tolerance, teacher setting, student model, optimizer updates,
 197 cosine schedule, checkpoint selection, and three training seeds. Each arm contains 4.4B supervised
 198 tokens and uses a peak learning rate of $4e-4$. Candidates are generated with the OpenHands scaffold.

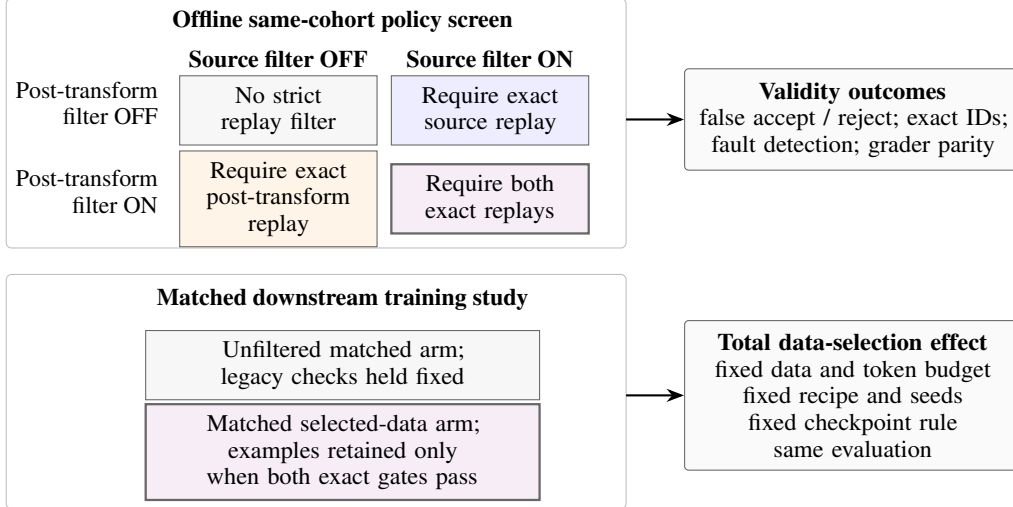


Figure 6: Experimental design. The 2×2 replay study compares policy filters on one frozen task cohort. The downstream study compares baseline and AutoBuild-selected data under matched training and evaluation budgets.

Table 1: Two-stage replay ablation on one frozen cohort. Invalid escape is the fraction of negative controls accepted; valid retention is the fraction of known-valid post-build tasks kept.

Acceptance policy	Invalid escape	Valid retention
No strict replay	14.6%	99.1%
Source replay R_s only	8.3%	88.7%
Grading replay R_g only	5.1%	79.4%
Full AutoBuild ($R_s + R_g$)	0.0%	68.2%

We evaluate 7B and 35B-A3B students on repository-disjoint held-out splits. NL2Repo-Bench is reported with Pass@1, while BeyondSWE-Doc2Repo is reported with mean test pass rate [3, 2]. The matched design estimates the total effect of AutoBuild data selection rather than the isolated training effect of each replay stage.

5 Results

5.1 Fresh Grading Removes Hidden-State Dependence

Benign candidates agree between the development container and the fresh grader in 99.2% of cases, indicating that isolation preserves behavior when the declared dependency contract is complete. Under controlled hidden-state dependencies, 93.8% of candidates pass in the reused work container but fail in the fresh grader. The same transition occurs for 7.1% of natural candidates. Identical exported artifacts always receive identical fresh-grader scores in our study. These results answer RQ1: replacing tests inside a mutable work container is not equivalent to evaluating the delivered repository, whereas the fixed grader restores artifact-determined scoring.

5.2 Two-Stage Replay Prevents Invalid Acceptance

Table 1 reports the before–after comparison and component ablation. Without strict replay, 14.6% of invalid controls escape the filter. Source replay alone reduces this rate to 8.3%, while grading replay alone reduces it to 5.1%. Combining both stages reduces invalid escape to 0.0%, a 14.6 percentage-point reduction, while retaining 68.2% of known-valid tasks. The lower retention is an explicit precision–coverage trade-off: strict validation removes tasks whose final evaluation path cannot be established, even when an earlier build checkpoint succeeded.

Table 2: Matched-budget downstream results with 4.4B supervised tokens per arm. Values are means over three seeds.

Student	Benchmark	Baseline	AutoBuild	Δ
7B	NL2Repo-Bench Pass@1	18.9	41.5	+22.6
7B	Doc2Repo mean pass	29.2	53.2	+24.0
35B-A3B	NL2Repo-Bench Pass@1	22.3	47.3	+25.0
35B-A3B	Doc2Repo mean pass	43.8	61.2	+17.4

The two stages are complementary. R_s primarily catches failures in reference setup, command extraction, and manifest construction; R_g exposes source-removal, candidate-cleanup, overlay, collection, and parser faults that occur only after task transformation. In the final accepted cohort, exact test-ID violations fall to 0; empty and import-only candidates are rejected in 100% of cases, behavior stubs in 96%, and source mutants in 90%. Adversarial and source-retrieval attacks escape in fewer than 2% of trials, and independent delivered graders agree on 99.7% of outcomes. These controls answer RQ2: validating only the reference build or only the final grader leaves distinct failure classes uncovered.

5.3 AutoBuild-Selected Data Improves Downstream Training

Table 2 reports the matched-budget training comparison. AutoBuild-selected data improves every model–benchmark pair. For the 7B student, NL2Repo-Bench Pass@1 increases from 18.9 to 41.5 (+22.6 points), while BeyondSWE-Doc2Repo mean test pass rate increases from 29.2 to 53.2 (+24.0 points). For the 35B-A3B student, the corresponding improvements are 22.3 to 47.3 (+25.0 points) and 43.8 to 61.2 (+17.4 points). Because the two arms match data volume, token budget, recipe, and evaluation, the comparison isolates the aggregate effect of selecting tasks and trajectories through the full validity contract.

Together, the task-validity and downstream results answer RQ3: the stricter filter does not merely reject malformed tasks; under a fixed training budget, the data it retains provide substantially stronger supervision for whole-repository generation.

6 Limitations and Broader Impacts

Scope. The implementation and experiments cover Python/pytest; other languages and test frameworks require their own identity parsers, cleanup rules, and fault suites. The replay ablation measures task validity, while the training study measures the full AutoBuild policy rather than separate downstream effects of R_s and R_g . Repository-level license manifests are not released.

Guarantees. Reference replay with negative controls is not a semantic oracle: it cannot prove specification completeness, test adequacy, or decontamination. Generator and grader can share mistakes, and a fresh judge can omit a legal dependency; held-out tests plus network, process, filesystem, and secret isolation remain necessary.

Broader impacts. Reliable labels reduce wasted compute and misleading capability claims. Repository harvesting nevertheless raises license, privacy, and supply-chain risks. Releases should preserve provenance and licenses, scan secrets, honor removals, and red-team source leakage and false rejection.

7 Conclusion

We introduced AutoBuild, a boundary-aligned repository-to-task synthesis and validation framework that validates tasks through the same transformation and judging path used at deployment. Clean grading makes scores depend on exported artifacts, while two-stage boundary-aligned replay, exact test identities, negative controls, and grader parity verify that task construction preserves the intended behavior. Full AutoBuild reduces invalid-control acceptance from 14.6% to 0.0% in our study, and AutoBuild-selected data consistently improves matched-budget training on NL2Repo-Bench and

BeyondSWE-Doc2Repo. These results support final-path validation as a necessary component of scalable, verifiable repository-task synthesis.

References

- [1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs, 2024. URL <https://arxiv.org/abs/2410.06992>.
- [2] Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, Ruihua Song, Chang Liu, Cheng Chen, Kai Jia, and Ji-Rong Wen. BeyondSWE: Can current code agent survive beyond single-repo bug fixing?, 2026. URL <https://arxiv.org/abs/2603.03194>.
- [3] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents, 2025. URL <https://arxiv.org/abs/2512.12730>.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- [5] Kenan Li, Rongzhi Li, Linghao Zhang, Qirui Jin, Liao Zhu, Xiaosong Huang, Geng Zhang, Yikai Zhang, Shilin He, Chengxing Xie, Xin Zhang, Zijian Jin, Bowen Li, Chaoyun Zhang, Yu Kang, Yufan Huang, Elsie Nallipogu, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. RepoLaunch: Automating build and management of code repositories across languages and platforms, 2026. URL <https://arxiv.org/abs/2603.05026>.
- [6] Haomin Qi, Xingliang Wang, Xuanqi Gao, Baihui Sang, Xin Zhang, Minghua Ma, Pengfei Gao, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Change2Task: From repository changes to executable coding agent tasks and environments, 2026. URL <https://arxiv.org/abs/2607.28591>.
- [7] Junlin Wang, Federico Bianchi, Shang Zhu, Fan Nie, Yongchan Kwon, Bhuwan Dhingra, and James Zou. Automated benchmark auditing for AI agents and large language models, 2026. URL <https://arxiv.org/abs/2605.26079>.
- [8] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/8b86cf5ace600c48fd188efbb8dedec8-Abstract-Datasets_and_Benchmarks_Track.html.
- [9] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Minghao Li, Jie Chen, Hui Xu, Yongshuai Sun, Wayne Xin Zhao, Ruihua Song, Yuan Zhang, Peng Wang, Cheng Chen, Ji-Rong Wen, and Kai Jia. Immersion in the GitHub universe: Scaling coding agents to mastery, 2026. URL <https://arxiv.org/abs/2602.09892>.
- [10] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Wayne Xin Zhao, Ruihua Song, Ji-Rong Wen, and Kai Jia. DeNovoSWE: Scaling long-horizon environments for generating entire repositories from scratch, 2026. URL <https://arxiv.org/abs/2606.10728>.

306 A Claims-to-Evidence Ledger

Claim	Evidence
Fresh grading removes hidden-state dependence	paired reused/fresh evaluation, controlled state injection, and natural-candidate transitions
Two replay stages are complementary	$R_s \times R_g$ ablation on one frozen task cohort
Full replay blocks invalid acceptance	invalid-control escape decreases from 14.6% to 0.0%
The final contract preserves test identity	exact collected/passed IDs, fail-closed return codes, negative controls, and grader parity
Selected data improve downstream capability	matched-budget 7B and 35B-A3B results on NL2Repo-Bench and Doc2Repo

307 B Boundary-Aligned Replay Procedure

```

308 for each repository snapshot r:
309     I_dev, C, parser, per_test = build_and_organize(r)
310     triage = execute_coarse(I_dev, C)
311     require triage.total > 0
312     require triage.passed / triage.total >= tau
313
314     ids = freeze(per_test.passing_ids)
315     require ids is nonempty
316     R_src = execute_exact(I_dev, C, ids)
317     require R_src.return_code == 0
318     require R_src.collected_ids == ids
319     require R_src.passed_ids == ids
320     require R_src.failed == R_src.errors == R_src.skipped == 0
321
322     doc = generate_spec(source=r.source, tests=r.tests,
323                        ast=inventory(r), coverage=profile(r))
324     I_grade, cleanup, manifest = package_clean_task(r, doc)
325
326     R_grade = deployed_judge(candidate=r.source,
327                             image=I_grade,
328                             cleanup=cleanup,
329                             tests=manifest)
330     require R_grade.return_code == 0
331     require R_grade.collected_ids == ids
332     require R_grade.passed_ids == ids
333     require R_grade.failed == R_grade.errors == R_grade.skipped == 0
334
335     for negative in [empty, import_stub, behavior_stub,
336                    mutants, adversarial_tests, source_fetch]:
337         require deployed_judge(negative).score < acceptance
338
339     require differential_parity(all_supported_graders)
340     persist image and command digests, parser hash,
341         test identities, outcomes, and gate decisions

```

342 C Evaluation and Reproducibility Details

343 Two-boundary replay adds approximately $1.7\times$ construction wall-clock relative to a legacy single
 344 build because each task executes exact source and post-transform replays plus negative controls. Each
 345 run stores immutable image and command digests, parser hashes, exact test identities and outcomes,
 346 process status, per-control decisions, and cross-grader outcomes for adjudication. The task-validity
 347 study applies every policy to the same frozen construction-complete cohort; the downstream study
 348 uses fixed benchmark and evaluator digests and a predeclared packing tolerance.

349 **D LLM Usage in the Core Method**

350 Language models are components of environment setup, test-interface organization, and specification
351 generation. The documentation agent uses repository browsing, test reading, symbol-source retrieval,
352 and runtime-introspection tools. The run manifest stores the model identifier, prompt hash, tempera-
353 ture, and seed for each generated artifact. Writing assistance for this manuscript is separate from the
354 scientific pipeline.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the problem formulation, the implemented protocol, and the validated outcomes; Sections 4 and 5 report the evaluation design, ablation, and downstream results, and Section 6 bounds generalization.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 discusses language and framework scope, the limits of reference replay, shared generator/grader failures, held-out testing, and license and release constraints.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper introduces definitions and target invariants, but makes no theorem or proof claim.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: Section 3 and Appendix B specify the protocol, and Section 4 reports the matched design, token budget, schedule, seeds, and evaluation procedure. The internal cohorts and run manifests are not released, so the submission does not provide an external path to fully reproduce every reported result.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The audited implementation and internal rollout data are not presently cleared for anonymous public release; Section 6 states this restriction.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [No]

Justification: Section 4 specifies the matched pool, equal 4.4B-token budget, 7B and 35B-A3B students, cosine schedule with peak $4e-4$, three seeds, fixed final-checkpoint selection, and pinned evaluators. Optimizer type, batch size, warmup, update count, exact checkpoint identifiers, and cohort sizes are not included in the present submission.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The downstream table reports means over three independent seeds with fixed final-checkpoint selection, but the present submission does not include per-seed dispersion or confidence intervals; the validity study likewise reports cohort rates without repository-clustered intervals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Section 4 fixes the 4.4B-token budget per arm and Appendix C reports the replay cost multiplier, but GPU type, worker count, memory, and end-to-end wall-clock are not included in the present draft.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This is a software-system audit with no human-subject experiments. The draft avoids exposing private repository locations or credentials and discusses responsible artifact release in Section 6.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 discusses both reduced label/compute waste and risks involving license compliance, privacy, supply chains, leakage, and false rejection.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This draft does not release a model or dataset. Proposed release safeguards are listed in Section 6.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Prior work is cited, but a repository-by-repository license and terms-of-use audit is not available for the internal task cohorts; no cohort assets are released with this draft.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper analyzes an internal prototype and introduces no publicly released dataset, model, or code asset in this submission draft.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or human-subject research and therefore requires no IRB approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Appendix D describes the language-model roles in environment and specification generation and the model-version, prompt-hash, temperature, and seed records stored per artifact.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.